

Grille d'autoévaluation - Semaine 1

420-W30-SF · Environnement .NET, Git, révision POO et diagrammes de classes · Automne 2026

Nom (facultatif)

Date de révision

Comment utiliser cette grille

Pour chaque énoncé, cochez la case seulement si vous pouvez accomplir l'action sans suivre une recette. Une case non cochée indique simplement une prochaine étape de révision. Conservez la grille et reprenez-la avant les évaluations.

1. Environnement de développement

Je suis capable de :

- ☐ distinguer une solution Visual Studio d'un projet;
- ☐ expliquer ce que décrit un fichier .csproj;
- ☐ créer une solution et y ajouter un projet;
- ☐ ajouter une référence d'un projet vers un autre;
- ☐ reconnaître le rôle du SDK, du runtime et du framework cible;
- ☐ compiler, exécuter et tester une solution avec les commandes dotnet;
- ☐ lire un diagnostic de compilation et localiser la première erreur utile;
- ☐ produire une sortie normale et une erreur avec Console.Out et Console.Error;
- ☐ expliquer pourquoi bin/ et obj/ ne sont pas des fichiers sources.

Preuve ou exemple que je peux montrer

2. Git de base

Je suis capable de :

- ☐ distinguer le répertoire de travail, la zone d'index et l'historique local;
- ☐ expliquer ce que montre git status;
- ☐ initialiser un dépôt Git;
- ☐ sélectionner explicitement une modification avec git add;
- ☐ créer un commit cohérent avec un message révélateur;
- ☐ consulter l'historique avec git log --oneline;
- ☐ expliquer pourquoi git add ne crée pas encore un commit;

- ☐ construire un .gitignore adapté à une solution .NET;
- ☐ vérifier qu'un fichier régénérable n'est pas suivi par erreur
- ☐ cloner un dépôt distant;
- ☐ expliquer la différence entre git pull et git fetch;
- ☐ pousser mes commits vers un dépôt distant;

Preuve ou commande que je peux exécuter sans aide

3. Vocabulaire de la POO

Je suis capable de :

- ☐ différencier une classe et un objet;
- ☐ différencier l'état, le comportement et l'identité d'un objet;
- ☐ reconnaître une variable d'objet, une variable de classe et une propriété C#;
- ☐ expliquer le rôle d'un constructeur;
- ☐ repérer l'objet qui possède l'information nécessaire à une règle;
- ☐ placer un comportement près des données qu'il protège;
- ☐ formuler un invariant simple d'un objet;
- ☐ expliquer comment une opération publique peut préserver ou briser un invariant.

4. Les quatre piliers

- ☐ Je peux citer les quatre piliers de la POO sans consulter mes notes.

1. Pilier 1

2. Pilier 2

3. Pilier 3

4. Pilier 4

- ☐ Je peux expliquer chacun des quatre piliers avec mes propres mots.
- ☐ Je peux donner un exemple C# pour chacun des quatre piliers.
- ☐ Je peux expliquer comment l'encapsulation protège la cohérence d'un objet.
- ☐ Je peux distinguer une abstraction de son détail concret.
- ☐ Je peux expliquer ce que promet une relation d'héritage.
- ☐ Je peux reconnaître un appel polymorphe à travers un contrat commun.
- ☐ Je peux expliquer pourquoi l'héritage ne sert pas seulement à réutiliser du code.

Pilier qui demande le plus de révision et raison

5. Abstractions, interfaces et composition

Je suis capable de :

- ☐ distinguer une classe abstraite d'une interface;
- ☐ expliquer ce qu'une interface promet à son client;
- ☐ écrire une interface C# étroite et deux implantations substituables;
- ☐ utiliser un objet à travers son interface;
- ☐ distinguer héritage et composition;
- ☐ reconnaître une situation où la composition est préférable;
- ☐ expliquer ce qui varie dans une collaboration entre objets;
- ☐ vérifier qu'une classe fille respecte les attentes du type parent.

6. Diagrammes de classes UML

Je suis capable de :

- ☐ représenter une classe avec son nom, ses membres et leur visibilité;
- ☐ interpréter les symboles +, -, # et ~;
- ☐ représenter une propriété C# avec ses accès pertinents;
- ☐ distinguer une association d'une dépendance;
- ☐ représenter une association dirigée;
- ☐ distinguer une agrégation d'une composition;
- ☐ expliquer le cycle de vie exprimé par une composition;
- ☐ représenter un héritage;
- ☐ représenter la réalisation d'une interface;
- ☐ ajouter des cardinalités seulement lorsqu'elles ont un sens;
- ☐ expliquer les cardinalités 1, 0..1, 0..* et 1..*;
- ☐ qualifier une relation avant de choisir sa flèche.

7. Passer du code au modèle et du modèle au code

Je suis capable de :

- ☐ produire un diagramme UML à partir de classes C#;
- ☐ écrire les classes C# correspondant à un diagramme UML;
- ☐ conserver les visibilité, les types et les cardinalités pertinentes;
- ☐ repérer une incohérence entre un diagramme et le code;
- ☐ expliquer les choix de relations de mon diagramme;

- ☐ compiler le code obtenu et corriger les écarts observés.

8. Défis de vérification

Sans consulter immédiatement les notes, je peux :

- ☐ créer une solution contenant un projet console, un projet de domaine et un projet de tests;
- ☐ modéliser Produit, LigneCommande et Commande avec leurs cardinalités;
- ☐ protéger une quantité strictement positive dans le domaine;
- ☐ expliquer pourquoi le calcul du sous-total appartient à LigneCommande;
- ☐ produire un petit historique Git composé de commits cohérents;
- ☐ expliquer oralement mon code et mon diagramme à une autre personne.

9. Mon plan de révision

Les trois notions que je dois revoir en priorité :

1.

2.

3.

La prochaine action concrète que je vais réaliser

Questions à apporter en classe